

Ejercicios de Listas, Tuplas, Conjuntos y Ordenamiento

[, , ,] Organiza, almacena y ordena tus datos

De PSeInt a Python — Guía Práctica Paso a Paso

Docen5te: Jhonathan Arley Peñaloza Sierra

Skill: Introducción a la Programación

Listas · Tuplas · Conjuntos · Algoritmos de Ordenamiento

Referencia Rápida: Lista vs Tupla vs Conjunto

Consúltalas cada vez que tengas dudas sobre cuál estructura usar.

Característica	Lista []	Tupla ()	Conjunto { }
Sintaxis	[1, 2, 3]	(1, 2, 3)	{ 1, 2, 3 }
¿Mutable?	Sí	No	Sí (add/remove)
¿Ordenada?	Sí	Sí	No
¿Duplicados?	Sí permite	Sí permite	No permite
¿Indexada?	Sí (desde 0)	Sí (desde 0)	No
Uso ideal	Datos que cambian	Datos fijos	Valores únicos

Recuerda: ¿Cuándo usar cada una?

Lista []: datos que necesitas modificar (agregar, eliminar, ordenar). Ejemplo: lista de tareas.

Tupla (): datos que NO deben cambiar. Ejemplo: coordenadas (x, y), días de la semana.

Conjunto { }: cuando necesitas valores únicos o eliminar duplicados. Ejemplo: correos únicos.

En PSeInt usabas Arreglos (Dimension). En Python tienes estas 3 opciones más poderosas.

PARTE 1: EJERCICIOS GUIADOS

Sigue cada paso como si estuvieras con un tutor a tu lado

Tema 1: Listas — Crear y Manipular

Una lista es un contenedor ordenado y mutable que almacena múltiples elementos. Cada elemento tiene un índice (posición) que empieza desde 0. En PSeInt usabas Arreglos con Dimension; en Python las listas son mucho más flexibles.

Ejercicio 1.1 — Crear y acceder a una lista

Enunciado

Crea una lista con 5 frutas. Muestra la fruta en el índice 0 (primera), la del índice 2 (tercera) y la última. Luego muestra cuántos elementos tiene la lista.

Guía paso a paso:

Paso 1: Crea la lista con corchetes [] y elementos separados por comas.

```
frutas = ["manzana", "pera", "naranja", "uva", "fresa"]
```

En PSeInt crearías un arreglo con: Dimension frutas[5]. En Python solo escribes los elementos entre corchetes. No necesitas declarar el tamaño.

Paso 2: Accede a los elementos por su índice.

```
print(frutas[0])    # manzana (primera)
print(frutas[2])    # naranja (tercera)
print(frutas[-1])   # fresa (última)
print(frutas[-2])   # uva (penúltima)
```

Los índices empiezan en 0. El índice -1 accede al último elemento, -2 al penúltimo, etc. En PSeInt los índices empezaban en 0 o 1 según la configuración.

Paso 3: Obtén la longitud de la lista con len().

```
print(f"Total de frutas: {len(frutas)}") # 5
```

Mini Reto 1.1

Muestra las primeras 3 frutas usando un slice: frutas[0:3].

Muestra las frutas desde la segunda hasta la última: frutas[1:].

Investiga: ¿Qué pasa si usas frutas[::-2]? (un paso de 2).

Solución sugerida — Ejercicio 1.1

```

frutas = ["manzana", "pera", "naranja", "uva", "fresa"]

print(frutas[0])      # manzana
print(frutas[2])      # naranja
print(frutas[-1])     # fresa
print(len(frutas))    # 5

# Mini reto
print(frutas[0:3])    # ["manzana", "pera", "naranja"]
print(frutas[1:])     # ["pera", "naranja", "uva", "fresa"]
print(frutas[:2])     # ["manzana", "naranja", "fresa"]

```

Explicación:

Los slices [inicio:fin] no incluyen el índice fin.

[::2] toma un elemento sí y uno no (paso de 2).

Las listas pueden mezclar tipos: [1, "hola", True, 3.14].

Ejercicio 1.2 — Agregar y eliminar elementos**Enunciado**

Crea una lista de frutas. Agrega "uva" al final con `append()`, inserta "fresa" en la posición 1 con `insert()`, elimina la primera fruta con `pop(0)`, y muestra la lista después de cada operación.

Guía paso a paso:

Paso 1: Crea la lista inicial y agrégala con `append()`.

```

frutas = ["manzana", "pera", "naranja"]
print(f"Original: {frutas}")

frutas.append("uva")
print(f"Después de append: {frutas}")
# ["manzana", "pera", "naranja", "uva"]

```

`append(elemento)` agrega UN elemento al FINAL de la lista. Es el método más usado para añadir elementos.

Paso 2: Inserta un elemento en una posición específica con `insert()`.

```

frutas.insert(1, "fresa")
print(f"Después de insert: {frutas}")
# ["manzana", "fresa", "pera", "naranja", "uva"]

```

`insert(posición, elemento)` inserta ANTES de la posición indicada. Los demás elementos se desplazan a la derecha.

Paso 3: Elimina con `pop()` y `remove()`.

```

eliminada = frutas.pop(0)    # Elimina y devuelve el primero
print(f"Se eliminó: {eliminada}")
print(f"Después de pop: {frutas}")

```

```
frutas.remove("pera")      # Elimina por valor (la primera que encuentre)
print(f"Después de remove: {frutas}")
```

pop(índice) elimina por posición y DEVUELVE el elemento eliminado. remove(valor) elimina la primera aparición del valor. Si el valor no existe, da error.

Paso 4: Agrega múltiples elementos con extend().

```
frutas.extend(["kiwi", "mango"])
print(f"Después de extend: {frutas}")
```

extend(lista) agrega TODOS los elementos de otra lista al final. No confundir con append(): append añade la lista como UN solo elemento.

append() vs extend() — ¡No los confundas!

lista.append([4, 5]) → [1, 2, 3, [4, 5]] ← Añade la lista COMO elemento.

lista.extend([4, 5]) → [1, 2, 3, 4, 5] ← Añade CADA elemento por separado.

Mini Reto 1.2

Usa clear() para vaciar la lista y verifica que queda vacía con print().

¿Qué pasa si usas pop() sin índice? Prueba y observa.

Solución sugerida — Ejercicio 1.2

```
frutas = ["manzana", "pera", "naranja"]
frutas.append("uva")      # agrega al final
frutas.insert(1, "fresa") # inserta en posición 1
frutas.pop(0)             # elimina el primero
frutas.remove("pera")     # elimina "pera"
frutas.extend(["kiwi", "mango"]) # agrega varios
print(frutas)
```

```
# pop() sin índice elimina el ÚTIMO elemento
ultimo = frutas.pop() # elimina y devuelve el último
```

Explicación:

append, insert, extend = agregar. pop, remove, clear = eliminar.

pop() devuelve el elemento eliminado; remove() no devuelve nada.

Estos métodos modifican la lista original (la mutan).

Ejercicio 1.3 — Buscar, contar y ordenar

Enunciado

Crea una lista de números desordenados. Busca si un número está en la lista con `in`, encuentra su posición con `index()`, cuenta cuántas veces aparece con `count()`, ordena la lista con `sort()`, e invértela con `reverse()`.

Guía paso a paso:

Paso 1: Crea la lista y busca con `in`.

```
numeros = [5, 3, 8, 1, 3, 9, 2, 3]

# Verificar si un elemento existe
print(8 in numeros)      # True
print(100 in numeros)    # False
```

El operador `in` devuelve `True` o `False`. Es la forma más simple de verificar si un elemento está en la lista.

Paso 2: Encuentra la posición y cuenta apariciones.

```
pos = numeros.index(8)    # ¿En qué posición está el 8?
print(f"El 8 está en el índice: {pos}") # 2

veces = numeros.count(3)  # ¿Cuántas veces aparece el 3?
print(f"El 3 aparece {veces} veces")    # 3
```

Paso 3: Ordena la lista.

```
numeros.sort()            # Ordena ascendente
print(numeros)            # [1, 2, 3, 3, 3, 5, 8, 9]

numeros.sort(reverse=True) # Ordena descendente
print(numeros)            # [9, 8, 5, 3, 3, 3, 2, 1]
```

`sort()` modifica la lista original. Si quieres una copia ordenada sin modificar la original, usa `sorted(lista)`. Python internamente usa el algoritmo `TimSort`.

Paso 4: Invierte el orden.

```
numeros.reverse()         # Invierte el orden actual
print(numeros)            # [1, 2, 3, 3, 3, 5, 8, 9]
```

Mini Reto 1.3

Crea una lista de palabras: `["manzana", "kiwi", "pera", "banana"]`.

Ordénalas alfabéticamente con `sort()`.

Ordénalas por longitud con `sort(key=len)`. ¿Cuál queda primera?

Solución sugerida — Ejercicio 1.3

```

numeros = [5, 3, 8, 1, 3, 9, 2, 3]
print(8 in numeros)          # True
print(numeros.index(8))      # 2
print(numeros.count(3))      # 3
numeros.sort()
print(numeros)               # [1,2,3,3,3,5,8,9]

# Mini reto
palabras = ["manzana", "kiwi", "pera", "banana"]
palabras.sort()              # alfabético
print(palabras)              # ['banana', 'kiwi', 'manzana', 'pera']
palabras.sort(key=len)       # por longitud
print(palabras)              # ['kiwi', 'pera', 'banana', 'manzana']

```

Explicación:

sort(key=len) ordena por la longitud de cada palabra. kiwi (4 letras) queda primera.
 sort() modifica la lista original. sorted() crea una NUEVA lista ordenada.

Ejercicio 1.4 — Listas anidadas (matrices)**Enunciado**

Crea una lista anidada 3×3 (como una tabla). Accede al elemento central (fila 1, columna 1).
 Recorre toda la matriz con un for anidado y muestra cada elemento.

Guía paso a paso:

Paso 1: Crea la matriz (lista de listas).

```

matriz = [
    [1, 2, 3],    # fila 0
    [4, 5, 6],    # fila 1
    [7, 8, 9]     # fila 2
]

```

Cada fila es una lista dentro de la lista principal. Esto crea una estructura tipo tabla de 3 filas por 3 columnas.

Paso 2: Accede a elementos con doble índice: matriz[fila][columna].

```

print(matriz[1][1])  # 5 (fila 1, columna 1 = centro)
print(matriz[0][2])  # 3 (fila 0, columna 2)
print(matriz[2][0])  # 7 (fila 2, columna 0)

```

Paso 3: Recorre toda la matriz con for anidado.

```

for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print() # salto de línea después de cada fila
# Resultado:

```

```
# 1 2 3
# 4 5 6
# 7 8 9
```

El for externo recorre cada fila. El for interno recorre cada elemento dentro de esa fila. `end=" "` hace que `print` no salte de línea.

Mini Reto 1.4

Calcula la suma de todos los elementos de la matriz usando el doble for.

Modifica el elemento central (fila 1, col 1) cambiándolo de 5 a 99.

Solución sugerida — Ejercicio 1.4

```
matriz = [[1,2,3],[4,5,6],[7,8,9]]
```

```
# Acceso
print(matriz[1][1]) # 5
```

```
# Suma total
suma = 0
for fila in matriz:
    for elem in fila:
        suma += elem
print(f"Suma total: {suma}") # 45
```

```
# Modificar el centro
matriz[1][1] = 99
print(matriz[1][1]) # 99
```

Explicación:

Las listas anidadas son muy útiles para representar tablas, tableros de juegos o imágenes. Siempre accedes con `[fila][columna]`. El primer índice es la fila, el segundo la columna.

Tema 2: Tuplas — Datos Inmutables

Una tupla es como una lista, pero INMUTABLE: una vez creada, no se puede modificar (no puedes agregar, eliminar ni cambiar elementos). Se usa para datos que no deben cambiar, como coordenadas o constantes.

Ejercicio 2.1 — Crear tuplas y desempaquetar

Enunciado

Crea una tupla con 5 ciudades de Colombia. Accede a la tercera ciudad. Intenta modificar un elemento (verás el error). Luego desempaqueta una tupla de coordenadas (x, y, z) en tres variables.

Guía paso a paso:

Paso 1: Crea la tupla con paréntesis ().

```
ciudades = ("Bogotá", "Medellín", "Cúcuta", "Cali", "Bucaramanga")
print(ciudades[2]) # Cúcuta (tercera, índice 2)
```

Las tuplas se crean con paréntesis () en vez de corchetes []. Se acceden con índice igual que las listas.

Paso 2: Intenta modificar un elemento (¡error!).

```
# ciudades[0] = "Cartagena"
# TypeError: 'tuple' object does not support item assignment
# Las tuplas SON INMUTABLES: no puedes cambiar sus elementos.
```

Paso 3: Desempaqueta valores en variables individuales.

```
coordenadas = (10, 20, 30)
x, y, z = coordenadas

print(f"x={x}, y={y}, z={z}") # x=10, y=20, z=30
```

Desempaquetar significa asignar cada valor de la tupla a una variable separada. El número de variables debe coincidir con el número de elementos.

Paso 4: Usa los métodos count() e index().

```
numeros = (1, 3, 5, 3, 7, 3)
print(numeros.count(3)) # 3 (aparece 3 veces)
print(numeros.index(5)) # 2 (está en el índice 2)
```

Las tuplas solo tienen 2 métodos: count() e index(). No tienen append, remove, sort, etc. porque son inmutables.

Truco: Convertir tupla a lista para modificar

Si necesitas modificar una tupla, conviértela a lista, modifica, y vuelve a tupla:

```
mi_lista = list(mi_tupla) # Tupla → Lista
mi_lista.append("nuevo") # Ahora sí puedes modificar
mi_tupla = tuple(mi_lista) # Lista → Tupla de vuelta
```

Mini Reto 2.1

Crea una tupla de un solo elemento: `mi_tupla = (42,)`. ¿Por qué necesita la coma?
 Prueba sin la coma: `mi_var = (42)`. ¿Qué tipo es? Usa `type()` para verificar.

Solución sugerida — Ejercicio 2.1

```
ciudades = ("Bogotá", "Medellín", "Cúcuta", "Cali", "Bucaramanga")
print(ciudades[2])          # Cúcuta

# Desempaquetar
x, y, z = (10, 20, 30)
print(f"x={x}, y={y}, z={z}")

# Mini reto
con_coma = (42,)            # ¡Es tupla!
sin_coma = (42)             # Es solo un int
print(type(con_coma))       # <class 'tuple'>
print(type(sin_coma))       # <class 'int'>
```

Explicación:

Sin la coma, Python interpreta `(42)` como el número 42 entre paréntesis, NO como tupla.
 Para una tupla de un solo elemento SIEMPRE necesitas la coma: `(elemento,)`.

Ejercicio 2.2 — Lista de tuplas: Librería**Enunciado**

Crea una lista de libros donde cada libro es una tupla (título, autor, género). Crea una función `buscar_por_genero(libros, genero)` que reciba la lista y un género, y retorne una lista con los títulos que coincidan.

Guía paso a paso:

Paso 1: Crea la lista de tuplas.

```
libros = [
    ("El Perfume", "Suskind", "Misterio"),
    ("La Metamorfosis", "Kafka", "Novela"),
    ("Misery", "Stephen King", "Misterio"),
    ("Harry Potter", "J.K. Rowling", "Novela")
]
```

Cada libro es una tupla (inmutable) porque sus datos no cambian. La lista de libros `libros` es mutable (puedes agregar o quitar libros). Esta combinación lista + tupla es muy común en programación.

Paso 2: Crea la función de búsqueda.

```
def buscar_por_genero(libros, genero):
    resultado = []
    for libro in libros:
        if libro[2] == genero:      # índice 2 = género
            resultado.append(libro[0]) # índice 0 = título
    return resultado
```

libro[0] = título, libro[1] = autor, libro[2] = género. El for recorre cada tupla, el if filtra por género, y append agrega el título al resultado.

Paso 3: Usa la función.

```
misterio = buscar_por_genero(libros, "Misterio")
print(misterio) # ["El Perfume", "Misery"]

novelas = buscar_por_genero(libros, "Novela")
print(novelas) # ["La Metamorfosis", "Harry Potter"]
```

Mini Reto 2.2

Crea una función `buscar_por_autor(libros, autor)` que busque por autor.
Agrega un nuevo libro a la lista y verifica que la búsqueda lo encuentre.

Solución sugerida — Ejercicio 2.2

```
libros = [
    ("El Perfume", "Suskind", "Misterio"),
    ("La Metamorfosis", "Kafka", "Novela"),
    ("Misery", "Stephen King", "Misterio"),
    ("Harry Potter", "J.K. Rowling", "Novela")
]
```

```
def buscar_por_genero(libros, genero):
    resultado = []
    for libro in libros:
        if libro[2] == genero:
            resultado.append(libro[0])
    return resultado
```

```
def buscar_por_autor(libros, autor):
    resultado = []
    for libro in libros:
        if libro[1] == autor:
            resultado.append(libro[0])
    return resultado
```

```
print(buscar_por_genero(libros, "Misterio"))
print(buscar_por_autor(libros, "Kafka"))
```

Explicación:

Lista de tuplas = patrón muy común para almacenar registros de datos.
Cada tupla es un "registro" con campos accesibles por índice.

Tema 3: Conjuntos (Sets) — Valores Únicos

Un conjunto (set) es una colección de elementos ÚNICOS y SIN ORDEN. Automáticamente elimina duplicados. Son ideales para verificar pertenencia rápidamente y para operaciones matemáticas como unión e intersección.

Ejercicio 3.1 — Crear conjuntos y eliminar duplicados

Enunciado

Crea un conjunto con números del 1 al 10 incluyendo repetidos y observa cómo elimina los duplicados. Luego agrega y elimina elementos usando `add()`, `remove()` y `discard()`. Finalmente, haz la unión de dos conjuntos.

Guía paso a paso:

Paso 1: Crea un conjunto y observa que elimina duplicados.

```
numeros = {5, 3, 8, 1, 3, 9, 5, 2, 1}
print(numeros)          # {1, 2, 3, 5, 8, 9} sin duplicados
print(type(numeros))    # <class 'set'>
```

Los conjuntos se crean con llaves `{ }`. Automáticamente eliminan elementos repetidos. Además, NO mantienen el orden en que los declaraste.

Paso 2: Agrega y elimina elementos.

```
numeros.add(10)          # Agrega el 10
print(numeros)

numeros.remove(3)        # Elimina el 3 (¡error si no existe!)
numeros.discard(100)     # Intenta eliminar 100 (NO da error)
print(numeros)
```

`remove()` lanza error si el elemento no existe. `discard()` hace lo mismo pero sin error. Usa `discard()` cuando no estés seguro si el elemento existe.

Paso 3: Unión de dos conjuntos.

```
pares = {2, 4, 6, 8, 10}
impares = {1, 3, 5, 7, 9}

todos = pares | impares    # Unión con |
print(todos)               # {1,2,3,4,5,6,7,8,9,10}

# También funciona con .union()
todos2 = pares.union(impares) # Mismo resultado
```

Uso práctico: eliminar duplicados de una lista

```
lista_con_duplicados = [1, 2, 2, 3, 3, 3, 4]
sin_duplicados = list(set(lista_con_duplicados))
```

```
# Resultado: [1, 2, 3, 4] (¡ojo: puede cambiar el orden!)
```

Mini Reto 3.1

Crea dos conjuntos de frutas con algunas en común.

Usa el operador & (intersección) para encontrar las frutas que están en AMBOS conjuntos.

Usa el operador - (diferencia) para encontrar las frutas que están SOLO en el primero.

Solución sugerida — Ejercicio 3.1

```
numeros = {5, 3, 8, 1, 3, 9, 5, 2, 1}
print(numeros)          # sin duplicados

numeros.add(10)
numeros.remove(3)
numeros.discard(100)    # sin error

pares = {2, 4, 6, 8}
impares = {1, 3, 5, 7}
print(pares | impares)  # unión

# Mini reto
a = {"manzana", "pera", "uva"}
b = {"pera", "kiwi", "uva", "mango"}
print(a & b)             # {'pera', 'uva'} (intersección)
print(a - b)            # {'manzana'} (solo en a)
```

Explicación:

| = unión (todos los elementos de ambos).

& = intersección (solo los que están en ambos).

- = diferencia (los que están en uno pero no en el otro).

Tema 4: Algoritmos de Ordenamiento

Los algoritmos de ordenamiento toman una lista y organizan sus elementos en orden. Python tiene `sort()` que usa TimSort internamente, pero entender cómo funcionan los algoritmos básicos es fundamental para pensar como programador.

Ejercicio 4.1 — Ordenamiento Burbuja (Bubble Sort)

Enunciado

Implementa el algoritmo de ordenamiento burbuja: compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite hasta que no haya más intercambios. Crea una función `burbuja(lista)` que ordene la lista.

Guía paso a paso:

Paso 1: Entiende la lógica: comparar vecinos e intercambiar.

```
# Ejemplo visual: [5, 3, 8, 1]
# Comparar 5 y 3: 5>3 → intercambiar → [3, 5, 8, 1]
# Comparar 5 y 8: 5<8 → no cambiar → [3, 5, 8, 1]
# Comparar 8 y 1: 8>1 → intercambiar → [3, 5, 1, 8]
# ... repetir hasta que esté ordenada
```

Paso 2: Implementa la función.

```
def burbuja(lista):
    n = len(lista)
    for i in range(n - 1):          # Pasadas
        for j in range(n - 1 - i):  # Comparaciones
            if lista[j] > lista[j + 1]: # Si están desordenados
                # Intercambiar
                lista[j], lista[j+1] = lista[j+1], lista[j]
    return lista
```

El for externo controla las pasadas (cada pasada "burbujea" el mayor al final). El for interno compara vecinos. La línea `a, b = b, a` es la forma de intercambiar en Python (sin variable temporal).

Paso 3: Prueba la función.

```
numeros = [5, 3, 8, 1, 9, 2]
print(burbuja(numeros))  # [1, 2, 3, 5, 8, 9]
```

Mini Reto 4.1

Modifica la función para que muestre la lista después de cada pasada (agrega un `print` dentro del for externo).

Agrega un parámetro `descendente=False`. Si es `True`, ordena de mayor a menor.

Solución sugerida — Ejercicio 4.1

```
def burbuja(lista, descendente=False):
    n = len(lista)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if descendente:
                if lista[j] < lista[j+1]:
                    lista[j], lista[j+1] = lista[j+1], lista[j]
            else:
                if lista[j] > lista[j+1]:
                    lista[j], lista[j+1] = lista[j+1], lista[j]
        print(f"Pasada {i+1}: {lista}")
    return lista
```

```
burbuja([5,3,8,1], False) # Ascendente
burbuja([5,3,8,1], True)  # Descendente
```

Explicación:

Burbuja es $O(n^2)$: lento para listas grandes, pero fácil de entender.
 En Python real, usa `.sort()` (TimSort). Burbuja es para APRENDER el concepto.
 El intercambio `a, b = b, a` es una característica elegante de Python.

Ejercicio 4.2 — Ordenamiento por Selección**Enunciado**

Implementa el algoritmo de selección: busca el elemento más pequeño en la parte no ordenada y colócalo al inicio. Repite con el resto. Crea una función `seleccion(lista)` que ordene la lista.

Guía paso a paso:

Paso 1: Entiende la lógica: encontrar el mínimo y colocarlo.

```
# [5, 3, 8, 1] → mínimo es 1 (pos 3) → intercambiar con pos 0
# [1, 3, 8, 5] → mínimo restante es 3 (pos 1) → ya está bien
# [1, 3, 8, 5] → mínimo restante es 5 (pos 3) → intercambiar con pos 2
# [1, 3, 5, 8] → ¡Ordenado!
```

Paso 2: Implementa la función.

```
def seleccion(lista):
    n = len(lista)
    for i in range(n - 1):
        indice_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[indice_min]:
                indice_min = j
        # Intercambiar el mínimo con la posición actual
        lista[i], lista[indice_min] = lista[indice_min], lista[i]
    return lista
```

Paso 3: Prueba.

```
print(seleccion([5, 3, 8, 1])) # [1, 3, 5, 8]
```

Solución sugerida — Ejercicio 4.2

```
def seleccion(lista):
    n = len(lista)
    for i in range(n - 1):
        indice_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[indice_min]:
                indice_min = j
        lista[i], lista[indice_min] = lista[indice_min], lista[i]
    return lista
```

```
print(seleccion([64, 25, 12, 22, 11])) # [11,12,22,25,64]
```

Explicación:

Selección hace pocos intercambios (máximo $n-1$), pero siempre recorre todo.
Es $O(n^2)$ como burbuja, pero con menos movimientos de datos.

PARTE 2: EJERCICIOS INDEPENDIENTES

Practica lo aprendido sin guía — Solo enunciado y datos de prueba

Listas

Ejercicio P1 — Lista de compras interactiva

Enunciado

Crea un programa que permita al usuario agregar productos a una lista de compras. El usuario escribe el nombre del producto y se agrega con `append()`. Cuando escriba "salir", muestra la lista completa y cuántos productos tiene.

Ejemplo de uso	Salida esperada
Producto: leche Producto: pan Producto: salir	Lista: ["leche", "pan"] Total: 2 productos

Pista: Usa `while True` con `break` cuando el usuario escriba "salir".

Ejercicio P2 — Invertir una lista sin `reverse()`

Enunciado

Pide 5 números al usuario y guárdalos en una lista. Luego crea una NUEVA lista con los elementos en orden inverso SIN usar `.reverse()` ni `[::-1]`.

Ejemplo de uso	Salida esperada
Números: 1, 2, 3, 4, 5	Original: [1,2,3,4,5] Invertida: [5,4,3,2,1]

Pista: Recorre la lista original desde el final con un `for` y `append()` cada elemento a la nueva lista.

Ejercicio P3 — Eliminar duplicados manualmente

Enunciado

Crea una lista con números repetidos. Sin usar `set()`, crea una nueva lista que contenga solo los elementos únicos (sin duplicados).

Ejemplo de uso	Salida esperada
<code>lista = [1,2,2,3,3,3,4,5,5]</code>	<code>[1, 2, 3, 4, 5]</code>

Pista: Recorre la lista y usa `not in` para verificar si el elemento ya está en la nueva lista.

Ejercicio P4 — Suma y promedio de una lista**Enunciado**

Pide al usuario N números (hasta que escriba 0). Calcula y muestra la suma total, el promedio, el máximo y el mínimo de la lista.

Ejemplo de uso	Salida esperada
<code>5, 10, 3, 8, 0</code>	Suma: 26, Promedio: 6.5 Máximo: 10, Mínimo: 3

Pista: Usa `sum(lista)`, `len(lista)`, `max(lista)` y `min(lista)`.

Ejercicio P5 — Matriz: suma de filas**Enunciado**

Crea una matriz 3×3 con números. Calcula y muestra la suma de cada fila.

Ejemplo de uso	Salida esperada
<code>matriz = [[1,2,3],[4,5,6],[7,8,9]]</code>	Fila 0: suma = 6 Fila 1: suma = 15 Fila 2: suma = 24

Pista: Usa un `for` para recorrer filas y `sum(fila)` para sumar cada una.

Tuplas

Ejercicio P6 — Catálogo de productos

Enunciado

Crea una lista de tuplas donde cada tupla es (nombre, precio, cantidad). Crea una función `valor_total(productos)` que calcule y retorne la suma de $\text{precio} \times \text{cantidad}$ de cada producto.

Ejemplo de uso	Salida esperada
<code>[("Pan", 2500, 3), ("Leche", 4000, 2)]</code>	Valor total: \$15500

Pista: Recorre con `for producto in productos` y acumula `producto[1] * producto[2]`.

Ejercicio P7 — Desempaquetar coordenadas

Enunciado

Crea una lista de tuplas con coordenadas (x, y). Recorre la lista, desempaqueta cada tupla en variables x, y, y calcula la distancia al origen con la fórmula: $d = (x^2 + y^2) ** 0.5$.

Ejemplo de uso	Salida esperada
<code>puntos = [(3,4), (0,5), (6,8)]</code>	<code>(3,4) -> distancia: 5.0</code> <code>(0,5) -> distancia: 5.0</code> <code>(6,8) -> distancia: 10.0</code>

Pista: Desempaqueta: `for x, y in puntos:`

Ejercicio P8 — Notas de estudiantes

Enunciado

Crea una lista de tuplas (nombre, nota). Crea funciones: `mejor_estudiante(notas)` que retorne el nombre con la nota más alta, y `aprobados(notas)` que retorne los nombres con $\text{nota} \geq 3.0$.

Ejemplo de uso	Salida esperada
<code>[("Ana",4.5),("Luis",2.8),("María",3.5)]</code>	Mejor: Ana (4.5) Aprobados: ["Ana", "María"]

Pista: Usa una variable para guardar el máximo actual mientras recorres.

Conjuntos

Ejercicio P9 — Correos únicos

Enunciado

Pide correos electrónicos al usuario (hasta que escriba "fin"). Guárdalos en un conjunto para evitar duplicados automáticamente. Al final muestra los correos únicos y cuántos hay.

Ejemplo de uso	Salida esperada
a@mail.com, b@mail.com, a@mail.com, fin	Correos únicos: {a@mail.com, b@mail.com} Total: 2

Pista: Usa un set() y .add() dentro de un while.

Ejercicio P10 — Estudiantes en común

Enunciado

Crea dos conjuntos: estudiantes de matemáticas y estudiantes de física. Encuentra: los que están en ambas materias (intersección), los que solo están en matemáticas (diferencia), y todos los estudiantes sin repetir (unión).

Ejemplo de uso	Salida esperada
mate={"Ana", "Luis", "María"} fis={"Luis", "Pedro", "María"}	Ambas: {"Luis", "María"} Solo mate: {"Ana"} Todos: 4 estudiantes

Pista: Usa los operadores & (intersección), - (diferencia), | (unión).

Ejercicio P11 — Letras únicas de una frase

Enunciado

Pide una frase al usuario. Convierte las letras a un conjunto para obtener las letras únicas (sin repetir, sin espacios). Muestra cuántas letras diferentes tiene la frase.

Ejemplo de uso	Salida esperada
frase: "hola mundo"	Letras únicas: {h,o,l,a,m,u,n,d} Total: 8 letras diferentes

Pista: Usa `set(frase.replace(" ", ""))` para eliminar espacios y obtener letras únicas.

Algoritmos de Ordenamiento

Ejercicio P12 — Inserción (Insertion Sort)

Enunciado

Implementa el algoritmo de inserción: toma cada elemento y lo inserta en su posición correcta comparando con los anteriores. Crea la función `insercion(lista)`.

Ejemplo de uso	Salida esperada
<code>insercion([5, 3, 8, 1])</code>	<code>[1, 3, 5, 8]</code>

Pista: El for externo va desde 1 hasta n. Guarda el elemento actual y desplaza los mayores hacia la derecha.

Ejercicio P13 — Comparar tiempos de ordenamiento

Enunciado

Crea una lista de 1000 números aleatorios. Ordena una copia con `burbuja()` y otra con `.sort()`. Muestra cuánto tarda cada uno (puedes contar las pasadas o usar `import time`).

Ejemplo de uso	Salida esperada
<code>import time</code> <code>lista = [random...]</code>	Burbuja: X.XX segundos <code>sort()</code> : X.XX segundos (<code>.sort()</code> será MUCHO más rápido)

Pista: Usa `import time` y `time.time()` para medir. Usa `import random` y `random.sample(range(10000), 1000)`.

Ejercicio P14 — Ordenar lista de tuplas

Enunciado

Crea una lista de tuplas (nombre, edad). Implementa burbuja modificado para ordenar por edad (de menor a mayor). No uses `.sort()`.

Ejemplo de uso	Salida esperada
<code>[("Ana", 25), ("Luis", 19), ("María", 22)]</code>	<code>[("Luis", 19), ("María", 22), ("Ana", 25)]</code>

Pista: En la comparación del burbuja, usa `lista[j][1] > lista[j+1][1]` para comparar por el segundo elemento (edad).

Ejercicio P15 — Ordenar y buscar

Enunciado

Crea una función que ordene una lista con cualquier algoritmo visto y luego otra función `busqueda_binaria(lista, objetivo)` que busque un número en la lista ordenada dividiendo a la mitad repetidamente. Retorna `True/False`.

Ejemplo de uso	Salida esperada
<code>busqueda_binaria([1,3,5,7,9], 5)</code>	<code>True</code>

Pista: Búsqueda binaria: compara con el medio. Si el objetivo es menor, busca en la mitad izquierda; si es mayor, en la derecha.

¡Felicidades por completar los ejercicios!

"Organizar bien tus datos es el primer paso para resolver cualquier problema"

Listas para datos que cambian. Tuplas para datos fijos. Conjuntos para valores únicos.

Y los algoritmos de ordenamiento para ponerlo todo en su lugar.